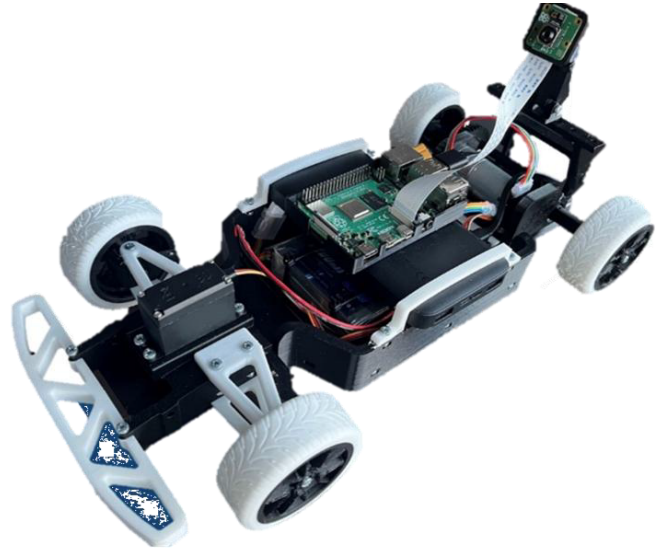


Modeling, Localization, Planning, and Control of an Ackermann Steering Autonomous Vehicle Using ROS2

Omar Ahmed Abdelatty, Youssef Omar Elkholy, Mohamed Ahmed Lotfy, Mazen Amr Mansour, Beshar Hezzeni, Michael Hany Samy
German University in Cairo (GUC), Egypt

Abstract—This project presents the implementation and simulation of an Ackermann steering autonomous ground vehicle using ROS2 and Gazebo. The developed system focuses on vehicle modeling, ROS2 communication, simulation validation, and vehicle motion control.

Keywords-ROS2, Gazebo, Autonomous Vehicle, Ackermann Steering



I. INTRODUCTION

Autonomous ground vehicles are a key research area in modern robotics due to their applications in transportation, logistics, and intelligent mobility. They integrate perception, localization, planning, and control to navigate without human intervention, requiring the ability to sense the environment, estimate state, plan safe paths, and execute motion commands [1].

Among ground vehicle steering mechanisms, the Ackermann configuration is the most widely used in conventional automotive systems. It allows the inner and outer wheels to follow different turning radii during cornering, reducing tire slip and improving maneuverability. Because of its realistic behavior, the Ackermann model is widely used in autonomous driving research and vehicle simulations [2].

In this work, a scaled autonomous ground vehicle based on the Ackermann steering principle is considered. The vehicle model used in the analysis is shown in Fig. 1. The system is implemented within the Robot Operating System (ROS2) framework, which provides a modular architecture where components communicate through message passing between nodes [1]. Additionally, the Gazebo simulation environment is used to model vehicle dynamics and visualize its interaction with the environment, allowing navigation and control algorithms to be tested safely before real-world deployment.

Fig. 1: Ackermann Steering Autonomous Vehicle Model Used in Simulation

The autonomous vehicle system analyzed in this report consists of several interconnected modules that together enable autonomous navigation. The localization module estimates the vehicle's position and orientation using sensor measurements. The navigation and planning module generates safe trajectories that guide the vehicle through the environment while avoiding obstacles. Finally, the control module regulates the vehicle velocity and steering angle to accurately follow the desired path [3].

This report presents a literature review of the key modules required for autonomous Ackermann-steered vehicles. Recent research on localization, path planning, and vehicle control methods is discussed, highlighting how these techniques can be integrated within a ROS-based autonomous driving framework. Understanding the interaction between these modules is essential for designing reliable autonomous ground vehicles capable of operating in complex environments.

II. LITERATURE REVIEW

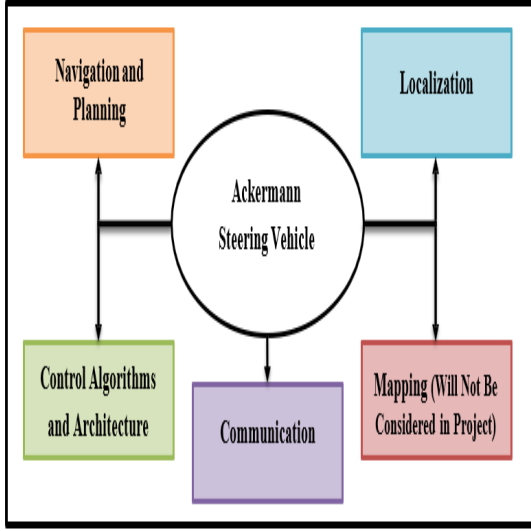


Fig. 2: Typical architecture of an autonomous vehicle system.

Autonomous vehicle systems typically use a modular architecture consisting of localization, planning, and control components. These modules enable reliable navigation in dynamic environments. Middleware frameworks such as ROS are commonly used to integrate perception, planning, and control algorithms while allowing different components to communicate through message passing interfaces [1].

A. Localization Methods

Localization is a critical component in autonomous navigation because the vehicle must estimate its position and orientation within the environment. Autonomous vehicles typically rely on multiple sensors such as LiDAR, cameras, GPS, and inertial measurement units (IMUs). Sensor fusion techniques are widely used to combine these measurements and improve localization accuracy. Probabilistic estimation methods such as Extended Kalman Filters (EKF) are commonly used to integrate sensor data and provide robust pose estimation [4]. In addition, LiDAR-based simultaneous localization and mapping (SLAM) algorithms enable vehicles to construct maps while estimating their position [5]. Visual-inertial approaches have also been explored to estimate vehicle motion in GPS-limited environments [6].

B. Navigation and Path Planning

The navigation module is responsible for generating feasible trajectories that guide the vehicle from its current position to a desired destination while avoiding obstacles. Path planning algorithms must consider environmental constraints, vehicle kinematics, and safety requirements.

Several planning strategies have been proposed in the literature for autonomous vehicles. Optimization-based planning approaches compute collision-free trajectories that satisfy vehicle dynamic constraints and minimize travel cost [2]. In addition, sampling-based algorithms and graph-based search

methods are commonly used for path generation in complex environments. Many ROS-based navigation frameworks implement real-time planners that continuously update the vehicle trajectory when new obstacles are detected [7]. These algorithms allow autonomous vehicles to adapt their motion in dynamic scenarios while maintaining safe operation.

C. Vehicle Control

The control module is responsible for executing the desired trajectory generated by the planning module while ensuring stable and accurate vehicle motion. Autonomous vehicles require both longitudinal control for regulating vehicle speed and lateral control for maintaining lane position.

Lane keeping control systems are widely studied for autonomous vehicles. Advanced lane keeping controllers use vehicle dynamic models and feedback control strategies to compute steering commands that maintain the vehicle within lane boundaries [3]. Model predictive control (MPC) techniques have also been proposed to improve trajectory tracking performance while respecting vehicle dynamic constraints [8]. Machine learning approaches such as recurrent neural networks have also been explored to improve steering control using sensor data [9].

III. GAZEBO VEHICLE MODELING AND SIMULATION

After validating the ROS2 workspace and communication framework, the Ackermann steering autonomous vehicle was integrated into the Gazebo simulation environment. Gazebo provided a realistic physics-based platform for validating vehicle motion and autonomous navigation algorithms without requiring physical hardware [10].

The simulated vehicle model was configured using the Ackermann steering mechanism, which accurately represents the steering geometry used in conventional autonomous ground vehicles. The simulation environment enabled testing of vehicle kinematics, steering behavior, and topic-based communication between ROS2 nodes and the vehicle model.

The Gazebo simulator was connected to ROS2 using the `ros_gz_bridge` package, which enabled bidirectional communication between Gazebo transport topics and ROS2 topics. Through this bridge, vehicle odometry and motion commands were exchanged successfully between the simulation environment and the ROS2 control nodes.

The vehicle model published odometry information through:

- `/model/vehicle_blue/odometry`

while velocity commands were transmitted using:

- `/model/vehicle_blue/cmd_vel`

The simulation framework also included localization, planning, lateral control, speed control, and constraint handling modules. These modules communicated using ROS2 publisher-subscriber mechanisms to generate and execute autonomous driving commands.

The developed simulation environment supports multiple testing scenarios including empty environments, racing tracks,

and structured road networks. This flexibility enables future implementation of advanced autonomous navigation algorithms such as lane following, obstacle avoidance, and trajectory tracking.

A. Simulation Tracks and Testing Environments

1) *Empty Track Environment*: The empty track environment consisted of a straight-line trajectory with a total length of approximately 10 meters. This simplified environment was used during the initial stages of system development to validate the basic operation of the autonomous vehicle framework before testing more complex driving scenarios.

The straight-line configuration enabled verification of:

- Vehicle spawning and Gazebo integration
- ROS2 topic communication
- Longitudinal speed control
- Straight-line trajectory tracking
- Odometry publishing and localization validation
- Steering stability during linear motion

The empty environment allowed analysis of vehicle motion without the influence of aggressive steering maneuvers or complex road geometries. This simplified setup was useful for validating the interaction between Gazebo, ROS2 nodes, and the vehicle control framework.

The 10-meter straight-line experiment was also used to analyze localization accuracy and filtering performance under stable driving conditions. Since the vehicle followed a linear trajectory with minimal steering input, the resulting localization signals contained lower dynamic disturbances compared to the racing and city track environments.

The obtained results confirmed stable vehicle motion, consistent odometry measurements, and successful communication between all implemented ROS2 modules.

2) *Racing Track Environment*: The racing track scenario introduced aggressive turns, continuous curvature changes, and higher-speed vehicle motion. This environment was used to evaluate the dynamic response of the lateral and longitudinal controllers under challenging trajectory conditions.

The racing environment tested:

- Vehicle trajectory tracking accuracy
- Steering controller responsiveness
- Localization stability during sharp turns
- Speed controller performance at higher velocities
- Path-following performance during continuous cornering maneuvers

The racing track produced larger localization disturbances because rapid steering changes increased vehicle heading variations and dynamic motion effects. Despite these conditions, the implemented filtering and control framework maintained stable vehicle operation and acceptable trajectory tracking performance.

3) *City Track Environment*: The city track environment simulated structured urban road conditions including intersections, lane changes, and moderate-speed driving scenarios. Compared to the racing track, the city environment emphasized stable navigation and smooth control performance.

This environment was used to validate:

- Lane-following behavior
- Smooth steering transitions
- Vehicle stability in structured road networks
- Localization consistency during urban navigation
- Planning and control module integration

The city environment generated smoother trajectories and smaller heading disturbances compared to the racing track. This allowed evaluation of localization accuracy and control performance under realistic autonomous driving conditions.

The use of multiple simulation environments improved the robustness of the developed framework and enabled testing under different operating conditions before future deployment on physical autonomous vehicle platforms.

IV. LOCALIZATION FILTERING AND STATE ESTIMATION

Accurate localization is essential for autonomous vehicle navigation because vehicle control and trajectory tracking algorithms depend on reliable position and orientation estimation. In practical autonomous systems, raw odometry measurements are typically affected by sensor noise, numerical disturbances, and small dynamic modeling errors.

To improve localization performance, a filtering stage was implemented to smooth noisy localization measurements and improve state estimation accuracy [11], [12].

The localization framework compared:

- Ideal vehicle states
- Noisy localization measurements
- Filtered localization estimates

The implemented filtering process reduced high-frequency noise components while preserving the overall vehicle trajectory behavior. This improved localization stability and provided smoother state estimates for the planning and control modules.

The localization filter significantly improved:

- Position estimation stability
- Heading angle consistency
- Trajectory smoothness
- Controller input reliability
- Vehicle tracking performance

Filtering is especially important during aggressive vehicle maneuvers such as sharp turns in the racing track scenario, where localization disturbances become larger due to rapid changes in steering angle and vehicle orientation.

The filtered localization estimates provided a more reliable representation of vehicle motion compared to the raw noisy signals, improving the overall stability of the autonomous vehicle framework.

V. ROS2 TOPICS AND COMMUNICATION ANALYSIS

The communication architecture of the autonomous vehicle system was analyzed using the ROS2 topic inspection tool `rqt_graph`. ROS2 topics enabled distributed message exchange between localization, planning, and control nodes within the autonomous driving framework.

Figure 3 illustrates the communication architecture implemented in the project.

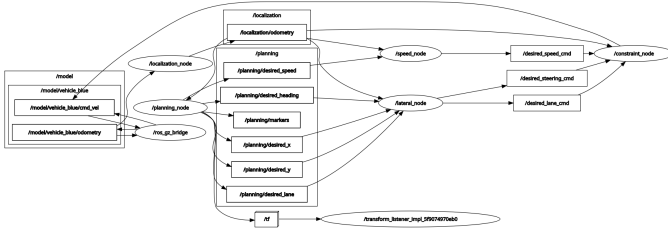


Fig. 3: ROS2 node and topic communication graph

The implemented architecture consisted of several interacting ROS2 nodes:

- /planning_node
- /localization_node
- /lateral_node
- /speed_node
- /constraint_node
- /ros_gz_bridge

The planning node generated desired trajectory references including desired position, heading angle, lane selection, and target speed. These references were published through the following topics:

- /planning/desired_x
- /planning/desired_y
- /planning/desired_heading
- /planning/desired_speed
- /planning/desired_lane

The localization node subscribed to vehicle odometry data published from Gazebo and provided vehicle state estimation to the control modules through:

- /localization/odometry

The lateral controller generated steering commands based on the desired trajectory and vehicle localization data. The speed controller regulated vehicle velocity using desired speed references.

The generated steering and speed commands were published through:

- /desired_steering_cmd
- /desired_speed_cmd

Finally, the constraint node processed the generated control commands and transmitted safe motion commands to the vehicle through Gazebo interfaces.

The implemented ROS2 communication framework verified successful publisher-subscriber communication between all autonomous driving modules and established the foundation required for future implementation of advanced navigation and control algorithms.

VI. VEHICLE MOTION CONTROL AND KINEMATICS

The control module is responsible for regulating vehicle motion and executing the desired trajectory generated by the

navigation system. Autonomous vehicles require both longitudinal and lateral control to maintain stable and accurate motion [3].

The Ackermann steering mechanism allows the inner and outer wheels to rotate at different steering angles during turning maneuvers. This improves vehicle stability and reduces tire slipping during cornering operations.

The kinematic bicycle model is commonly used to represent Ackermann steering vehicles [13]. The vehicle motion can be described using the following equations:

$$\dot{x} = v \cos(\theta) \quad (1)$$

$$\dot{y} = v \sin(\theta) \quad (2)$$

$$\dot{\theta} = \frac{v}{L} \tan(\delta) \quad (3)$$

where:

- x and y represent vehicle position
- θ represents vehicle heading angle
- v represents vehicle velocity
- L represents wheelbase length
- δ represents steering angle

The implemented control architecture enables future integration with autonomous navigation and trajectory tracking algorithms. These equations provide the mathematical foundation required for localization, path planning, and vehicle control [8].

The ROS2 communication framework allows steering commands and velocity commands to be transmitted directly to the simulated vehicle through motion control topics [1].

VII. SIMULATION RESULTS AND DISCUSSION

The developed ROS2 and Gazebo autonomous vehicle framework was successfully validated through multiple simulation experiments. All ROS2 packages compiled correctly using the `colcon` build system, and the implemented nodes executed successfully during runtime.

The Gazebo simulation environment successfully launched the Ackermann steering vehicle model and established communication with ROS2 through the `ros_gz_bridge` interface. Vehicle odometry, velocity commands, and coordinate transformations were transmitted correctly between Gazebo and ROS2 nodes.

The `rqt_graph` analysis verified successful communication between the localization, planning, lateral control, speed control, and constraint handling modules through ROS2 publisher-subscriber communication.

The planning module generated desired vehicle states including position, heading angle, lane selection, and vehicle speed, while the lateral and longitudinal controllers generated steering and velocity commands for trajectory tracking.

The simulation framework was tested using three Gazebo environments:

- **Empty Track:** A 10-meter straight-line environment used to validate ROS2 communication, odometry transmission, and straight-line trajectory tracking.

- **Racing Track:** A high-curvature track used to evaluate localization and controller performance during aggressive steering maneuvers.
- **City Track:** A structured urban environment used to test smooth lane-following behavior and localization stability under realistic road conditions.

The localization framework compared ideal vehicle states, noisy measurements, and filtered localization estimates. The implemented filter successfully reduced localization noise and improved trajectory tracking performance.

A. Racing Track Results

The racing track experiments produced larger localization disturbances due to aggressive steering maneuvers and continuous heading angle variations.

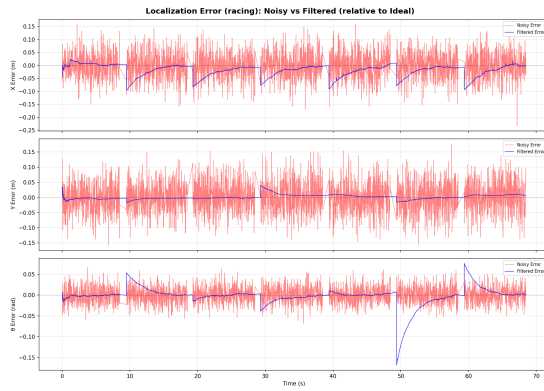


Fig. 4: Localization error comparison in the racing track environment

The noisy localization signals contained high-frequency disturbances commonly observed in practical localization systems [11]. After applying the localization filter, the resulting signals became smoother and more stable, improving controller performance during sharp cornering maneuvers.

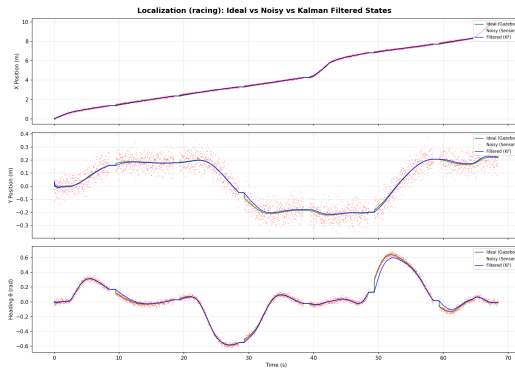


Fig. 5: Vehicle localization states in the racing track environment

The figure shows the filtered vehicle states during racing track navigation. The localization filter reduced signal oscillation

and generated smoother position and heading estimates during aggressive steering maneuvers.

Figure 6 illustrates the vehicle trajectory in the racing track environment.

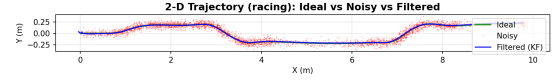


Fig. 6: Vehicle trajectory tracking in the racing track environment

The obtained trajectory demonstrates successful path-following performance despite the challenging racing conditions. Small trajectory deviations appeared mainly during rapid steering transitions.

B. City Track Results

The city track experiments demonstrated smoother vehicle motion and improved localization stability due to lower vehicle speeds and less aggressive steering behavior.

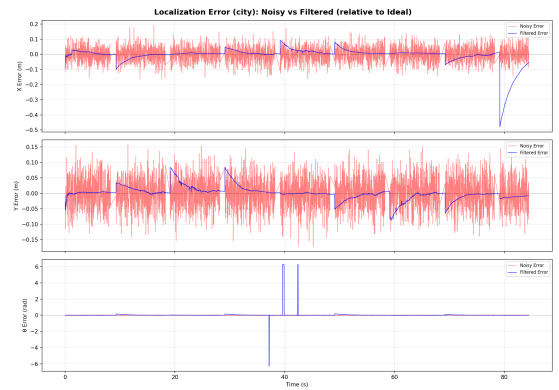


Fig. 7: Localization error comparison in the city track environment

Compared to the racing scenario, the city track generated smaller localization disturbances and smoother heading variations. The filtering stage successfully reduced noise and produced stable position and orientation estimates.

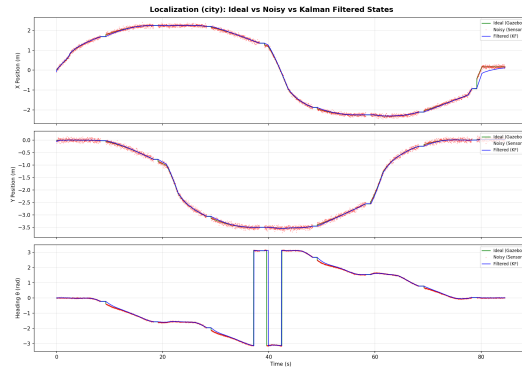


Fig. 8: Vehicle localization states in the city track environment

The city track generated smoother localization states and smaller heading variations compared to the racing scenario. The filtered signals closely followed the ideal vehicle states.

Figure 9 shows the trajectory tracking performance in the city track environment.

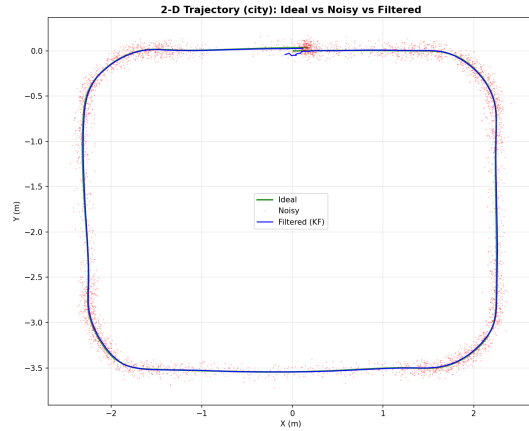


Fig. 9: Vehicle trajectory tracking in the city track environment

The vehicle maintained smooth motion while following the urban driving trajectory with acceptable localization accuracy and reduced tracking disturbances compared to the racing environment.

Overall, the obtained results verified successful integration between Gazebo, ROS2 communication modules, localization filtering, planning algorithms, and vehicle controllers. The developed framework provides a suitable foundation for future implementation of autonomous navigation, obstacle avoidance, and advanced trajectory tracking algorithms.

VIII. CONCLUSION

This project presented the implementation and simulation of an Ackermann steering autonomous vehicle using ROS2 and Gazebo. The ROS2 workspace was successfully configured and validated through the implementation of custom ROS2 nodes and communication interfaces [1].

The Gazebo simulation environment enabled visualization of vehicle motion and provided a realistic testing framework for autonomous vehicle development. Communication

between ROS2 nodes and the simulated vehicle was successfully achieved using ROS2 topics and publisher-subscriber communication mechanisms.

The implemented system established the foundation required for future development of localization, path planning, obstacle avoidance, and autonomous navigation algorithms. The developed framework can also be extended toward real hardware implementation and advanced autonomous driving applications [2].

Future work may include implementation of SLAM algorithms, real-time obstacle detection, sensor fusion techniques, and advanced trajectory tracking controllers for fully autonomous operation [4], [5], [8].

REFERENCES

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Ng, "Ros: An open-source robot operating system," *ICRA Workshop on Open Source Software*, 2009.
- [2] Y. Liu and M. Tomizuka, "Trajectory planning for autonomous vehicles in dynamic environments," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 8, pp. 5003–5015, 2021.
- [3] H. Wang, B. Liu, and J. Qiao, "Advanced high-speed lane keeping system of autonomous vehicle with sideslip angle estimation," *Machines*, vol. 10, no. 4, p. 257, 2022.
- [4] A. Martinez, J. Morales, and R. Gonzalez, "A multi-sensor fusion localization system for autonomous vehicles," *Sensors*, vol. 23, no. 4, p. 1890, 2023.
- [5] F. Aghili, "Lidar-based simultaneous localization and mapping for autonomous vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 7, no. 2, pp. 215–227, 2022.
- [6] X. Zhang, H. Li, and Y. Chen, "Map-aided visual-inertial fusion localization method for autonomous driving vehicles," *Measurement*, vol. 210, p. 112548, 2023.
- [7] R. M. K. Rahul, S. Ramesh, and P. Kumar, "Autonomous vehicle navigation with lidar using path planning," *IEEE Access*, vol. 10, pp. 56 780–56 791, 2022.
- [8] J. Ji, A. Khajepour, and W. Melek, "Model predictive control for autonomous vehicle path tracking," *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 4, pp. 725–735, 2021.
- [9] Y. Jeong, "Interactive lane keeping system for autonomous vehicles using lstm-rnn considering driving environments," *Sensors*, vol. 22, no. 24, p. 9610, 2022.
- [10] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2004, pp. 2149–2154.
- [11] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [12] A. Kelly, "A 3-d state space formulation of a navigation kalman filter for autonomous vehicles," Carnegie Mellon University Robotics Institute, Tech. Rep., 1994.
- [13] R. Siegwart, I. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011.